Architecture des Systèmes d'Exploitation : Processus, Ordonnancement et Signaux POSIX

Thème principal : Gestion des processus, cycle de vie (Élu/Prêt/Bloqué), arborescence PID/PPID, appels système fork()/exec() et signaux POSIX

Synthèse exécutive : Ce cours pose les fondements de l'architecture des systèmes d'exploitation de type Unix/Linux (module R1.04). Il détaille le passage crucial d'un programme statique (fichier exécutable résidant sur le disque dur) à un processus dynamique (instance active chargée en mémoire RAM). Pour assurer un multitâche préemptif fluide sur des ordinateurs à ressources partagées, l'ordonnanceur (scheduler) du noyau gère le passage continuel de chaque processus à travers un cycle de vie strict à trois états : Élu (Running), Prêt (Ready) et Bloqué (Waiting/Blocked). L'analyse de l'arborescence généalogique des processus met en évidence le rôle central du processus initial systemd (PID 1), créateur direct ou indirect de tous les autres processus via les appels système fork() (duplication) et exec() (recouvrement mémoire). Enfin, la communication et le contrôle inter-processus (IPC) s'appuient sur les signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT), des interruptions logicielles asynchrones permettant la gestion propre ou l'arrêt forcé d'applications.

1. Du Programme Statique au Processus Dynamique : Cycle de Vie, PCB et Multitâche

Pour qu'un ordinateur exécute des tâches, le système d'exploitation doit transformer un programme (ensemble d'instructions mortes sur le disque dur) en une instance active appelée processus, chargée en mémoire vive (RAM). Pour gérer plusieurs applications en même temps, le noyau (kernel) utilise le multitâche préemptif : l'ordonnanceur (scheduler) attribue successivement le processeur (CPU) aux processus pendant de très courts intervalles (quanta de temps). Au cours de son existence, un processus traverse obligatoirement trois états : 1. Élu (Running) : Le processus occupe le processeur et exécute activement ses instructions machine. 2. Prêt (Ready) : Le processus possède toutes ses ressources en mémoire RAM mais attend que l'ordonnanceur lui alloue du temps processeur. 3. Bloqué (Waiting/Blocked) : Le processus est suspendu car il a demandé une opération externe lente (lecture sur disque, saisie au clavier, réception d'un paquet réseau). Pour éviter de gâcher du temps CPU, le noyau le retire du processeur. Une fois l'opération terminée, le processus ne repasse PAS immédiatement en Élu : il retourne obligatoirement dans l'état Prêt avant d'être à nouveau choisi par l'ordonnanceur. Tous les processus s'organisent selon une arborescence généalogique stricte. Au démarrage du système, le noyau initialise le premier processus utilisateur : systemd (ou init), recevant le PID 1. Pour lancer une commande (ex: dans un terminal Bash), le processus parent utilise l'appel système fork() pour se cloner. Le fils hérite d'un nouveau PID et du PPID de son père. Il utilise ensuite l'appel système exec() pour vider la mémoire héritée et charger le code du nouveau binaire à exécuter.

- • Programme vs Processus : Un programme est une suite statique d'instructions binaire stockée sur disque. Un processus est une entité dynamique en exécution dans la mémoire RAM possédant des ressources système.
- • Multitâche Préemptif & Quantum de temps : Technologique du noyau Linux attribuant de minuscules tranches de temps CPU aux processus éligibles afin de simuler l'exécution parallèle simultanée de plusieurs programmes.
- • PCB (Process Control Block) : Structure de données interne du noyau qui conserve toutes les métadonnées d'un processus : état du CPU, pointeurs mémoire (Stack, Heap, Code, Data), PID, PPID, UID et descripteurs de fichiers.
- • Arborescence PID/PPID : Filiation généalogique où chaque processus est identifié par un PID unique et relié à un parent via son PPID. Le processus racine utilisateur est systemd (ou init) qui possède toujours le PID 1.
- • Mecanisme fork() et exec() : fork() duplique à l'identique un processus parent pour engendrer un processus fils ; exec() remplace le code mémoire de ce fils par le binaire d'un nouveau programme.
- • Cycle de vie à 3 états : Élu (occupe le CPU), Prêt (attend son quantum CPU), Bloqué (en attente d'une entrée/sortie disque, réseau ou clavier).

Cas concret enseignant : *L'analogie du livre de recettes de cuisine : Le professeur compare un programme sur disque dur à un livre de recettes rangé dans une armoire. Il ne consomme ni électricité, ni ingrédients, ni temps. Le processus est le cuisinier qui exécute la recette dans sa cuisine : il occupe de l'espace sur le plan de travail (la mémoire RAM), consomme du temps d'attention et d'énergie (le CPU) et manipule des ustensiles (les descripteurs de fichiers).*

2. Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX représentent le mécanisme le plus simple et universel de communication inter-processus (IPC - Inter-Process Communication). Un signal est une interruption logicielle envoyée à un processus de manière asynchrone : sa réception interrompt immédiatement le cours normal des instructions de la cible. À la réception d'un signal standard, un processus a trois options : 1. Exécuter l'action par défaut (souvent la fermeture immédiate du programme). 2. Ignorer purement et simplement le signal. 3. Intercepter le signal en le capturant via une fonction dédiée (Signal Handler). Cela permet par exemple à un éditeur de texte de bloquer le signal d'interruption pour demander à l'utilisateur s'il veut sauvegarder son fichier avant de fermer. Cependant, afin de garantir au système et à l'administrateur le pouvoir absolu de reprendre la main sur une application plantée ou malveillante, la norme POSIX impose deux exceptions incontournables : SIGKILL (Signal 9) et SIGSTOP (Signal 19). Ces signaux ne parviennent jamais au code de l'application utilisateur : ils sont interceptés et exécutés en priorité absolue directement par le noyau Linux. Un programme ne peut donc en aucun cas les intercepter ou les éviter.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus pour lui notifier un événement ou lui ordonner une action.
- Gestionnaire de signal (Signal Handler) : Fonction logicielle programmée pour intercepter (catch) un signal et remplacer le comportement système par défaut par un traitement personnalisé.
- SIGINT (Signal 2) : Interruption douce générée par la combinaison 'Ctrl + C' depuis le terminal.
- SIGTERM (Signal 15) : Demande de terminaison propre transmise par défaut par la commande 'kill ', permettant d'enregistrer et de libérer les ressources.
- SIGKILL (Signal 9) : Ordre d'arrêt inconditionnel traité directement par le noyau Linux. Il ne peut être ni intercepté, ni ignoré, ni masqué.
- SIGSTOP (Signal 19) et SIGCONT (Signal 18) : Raccourci 'Ctrl + Z' envoyant SIGSTOP pour suspendre un processus ; les commandes 'fg' ou 'bg' envoient SIGCONT pour en relancer l'exécution.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à la porte pour vous demander de ranger vos affaires et de sortir (vous pouvez fermer le gaz, sauvegarder vos fichiers et verrouiller la porte). SIGKILL (Signal 9) est une intervention de la brigade de sécurité qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : La commande kill attend que le numéro de signal soit précédé d'un tiret (ex: -9 ou -SIGKILL) pour le traiter comme une option. Si vous écrivez 'kill 9 1234', kill interprète '9' comme étant le premier PID à détruire, et '1234' comme un second PID, tout en appliquant le signal par défaut SIGTERM (15). Le terminal renvoie une erreur car le PID 9 est soit inexistant soit réservé au noyau, ce qui empêche d'envoyer le signal au PID 1234.

Question étudiant : Qu'est-ce qu'un processus 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus Zombie est un processus fils qui a totalement terminé son exécution, mais dont le père n'a pas encore lu la valeur de retour via l'appel système wait(). Il ne consomme plus de CPU ni de RAM, mais son entrée reste enregistrée dans la table des processus du noyau. On NE PEUT PAS détruire un Zombie avec la commande 'kill -9' car le processus est DÉJÀ mort ! Pour faire disparaître un Zombie, il faut soit forcer le père à exécuter wait(), soit tuer le processus père : le Zombie est alors adopté par systemd (PID 1) qui nettoie immédiatement l'entrée.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant tape la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le traitement réalisé lors de l'envoi du signal -15 et décrivez le résultat de la vérification ultérieure par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' situé en fin de commande indique au shell d'exécuter la tâche 'sleep 600' en arrière-plan (background). Le shell attribue immédiatement un job ID (ex: [1]), affiche le PID affecté par le noyau (ex: 4521) et redonne tout de suite l'invite de commande (prompt) à l'utilisateur sans bloquer le terminal. Étape 2 : La commande 'ps -ef | grep sleep' interroge la table des processus en cours d'exécution sur le système (-ef) et filtre la liste avec 'grep' pour ne conserver que la ligne contenant la chaîne 'sleep'. Cela permet d'identifier formellement le PID du processus (ex: 4521) ainsi que son PPID (le PID du shell Bash). Étape 3 : La commande 'kill -15 4521' envoie le signal SIGTERM (Signal 15) au processus. Comme la commande 'sleep' n'a pas réécrit de gestionnaire de signal particulier pour intercepter SIGTERM, l'action par défaut du système s'applique : le processus s'arrête proprement et libère toutes ses ressources système.

Résultat : Le processus 'sleep 600' est détruit de manière propre et ordonnée par le signal SIGTERM (15). Une exécution de contrôle via 'ps -ef | grep sleep' confirme la suppression définitive du processus dans la table du noyau.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux lors de l'initialisation du système ?

R : Il s'agit du processus 'systemd' (ou 'init' sur d'anciennes distributions), et son PID est toujours 1.

Q : Quel signal POSIX est transmis à une application en cours d'exécution lorsqu'un utilisateur saisit la combinaison de touches 'Ctrl + C' dans le terminal ?

R : Il s'agit du signal SIGINT (Signal 2), qui demande une interruption douce du processus interactif.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté, masqué ou ignoré par un programme utilisateur ?

R : Parce qu'il n'est pas transmis au code du programme : SIGKILL est capturé et exécuté directement au niveau du noyau (kernel) pour garantir au système la possibilité absolue d'arrêter tout processus.